

Multiple Choice Quiz

Distributed Systems: Mutual Exclusion, Snapshots, ZooKeeper & Kafka

Instructions: For each of the following 10 questions, select the **best** answer from the four options (A–D). These questions cover classic distributed mutual exclusion algorithms, consistent snapshot techniques, and modern coordination services (ZooKeeper & Kafka). There is exactly one correct answer per question.

Question 1. Why is implementing shared-resource locking difficult in a distributed system compared to a single-machine environment?

- A) Single-machine environments rely exclusively on hardware spinlocks while distributed systems are restricted to software-only semaphores.
 - B) Network bandwidth limitations are the dominant factor; the actual locking logic itself is trivial once bandwidth is sufficient.
 - C) Because every process in a distributed system must be written in the exact same programming language to use compatible lock primitives.
 - D) Distributed systems lack shared memory and a global clock, making it impossible to use traditional local locking mechanisms. Furthermore, messages can be lost or delayed, and processes can crash, requiring robust algorithms to ensure safe resource access.
-

Question 2. What is the primary trade-off when using a centralized approach for mutual exclusion?

- A) Centralized locking guarantees perfect fairness among processes but requires synchronized physical clocks on all nodes.
 - B) The centralized approach is simple to implement because a single coordinator grants all locks. However, it suffers from being a single point of failure; if the coordinator crashes, the entire locking system fails.
 - C) It achieves the lowest possible message complexity but at the cost of very high latency for every lock acquisition.
 - D) The approach has extremely high message overhead compared with fully distributed algorithms such as Ricart-Agrawala.
-

Question 3. How does the Token-ring approach for mutual exclusion ensure fairness?

- A) Every requesting process must obtain approval from a majority of other processes before entering the critical section.
 - B) A dynamically elected leader fairly rotates access rights among all processes that have expressed interest in the resource.
 - C) It uses Lamport timestamps and always awards the lock to the process with the smallest timestamp among waiting requesters.
 - D) In this approach, a token circulates through the processes in a logical ring, and only the holder of the token can access the resource. This ensures fairness and prevents starvation, though the loss of the token is a significant failure state.
-

Question 4. What is the message complexity of the Ricart-Agrawala algorithm per request?

- A) It requires only $O(\log N)$ messages in the common case thanks to a hierarchical grouping of processes.
 - B) N messages are sent in the request phase and another N messages are sent when the process releases the resource.
 - C) The Ricart-Agrawala algorithm is a fully distributed, timestamp-based voting approach. It requires $2(N-1)$ messages for every request to enter a critical section, where N is the number of processes.
 - D) Exactly three messages are exchanged per request regardless of the number of processes in the system.
-

Question 5. What is the "Bully algorithm" approach to leader election?

- A) The Bully algorithm is a classic leader election method based on process identifiers. In this "intuition-only" model, the process with the highest unique ID is elected as the leader.
 - B) Processes exchange their current physical clock values and the one with the most advanced time wins the election.
 - C) A central coordinator process periodically polls all others and designates the healthiest one as the new leader.
 - D) It circulates a special token; the first stable process that receives the token becomes the permanent leader.
-

Question 6. Why can taking local snapshots at different wall-clock times lead to inconsistent results?

- A) Different processes may be running incompatible snapshot algorithms that produce data in incompatible formats.
 - B) Local snapshots always miss messages that have already been sent but have not yet arrived at the destination process.
 - C) Because messages are often in flight, taking snapshots at different wall-clock times can make money or data appear to disappear or double-count. This occurs because the snapshots do not represent a single logical moment across the entire distributed system.
 - D) Wall-clock time is not monotonic; NTP adjustments can cause a later snapshot to record an earlier logical state.
-

Question 7. What is the primary role of "marker" messages in the Chandy-Lamport snapshot algorithm?

- A) They are used to detect and silently discard duplicate or out-of-order messages during the snapshot recording phase.
 - B) Markers carry the current Lamport timestamp so every process can align its snapshot to exactly the same logical time.
 - C) Markers are sent on all outgoing channels to initiate a snapshot; when a process receives its first marker, it records its local state. It then begins recording messages on other channels until markers have been received on all channels, completing the consistent cut.
 - D) Markers are broadcast globally to elect a temporary snapshot coordinator that collects state from every process.
-

Question 8. What was the original goal of the ZooKeeper service developed by Yahoo?

- A) To implement a fully ACID-compliant distributed transactional database across multiple Yahoo datacenters.
 - B) To serve as a high-performance, scalable replacement for the Hadoop NameNode metadata store.
 - C) To provide a distributed file system optimized for Yahoo's massive web-search index storage requirements.
 - D) ZooKeeper was originally developed at Yahoo to provide reliable coordination for large-scale distributed systems. It was designed to handle tasks like task assignment and configuration management.
-

Question 9. What are the two simple but powerful primitives used by ZooKeeper to manage coordination?

- A) Ephemeral znodes for leader election and sequential znodes for distributed locking and queuing.
 - B) Barriers for synchronizing process startup and distributed queues for reliable task hand-off between workers.
 - C) ZooKeeper utilizes "znodes" (a hierarchical namespace similar to a file system) and "watches" (a mechanism to notify clients of changes). Together, these allow developers to build complex coordination logic.
 - D) Heart-beat messages for failure detection combined with automatic leader election among healthy nodes.
-

Question 10. How has the coordination architecture of Apache Kafka evolved in recent versions?

- A)** Kafka has always used its own internal consensus protocol; ZooKeeper was only required by very old client libraries.
 - B)** Kafka removed all built-in coordination and now requires users to deploy and manage a separate etcd cluster.
 - C)** It replaced its original Raft implementation with ZooKeeper to achieve better scalability and operational simplicity.
 - D)** Historically, Kafka relied on ZooKeeper for managing its coordination and metadata. In modern implementations, Kafka has largely moved away from ZooKeeper in favor of KRaft (Kafka Raft), its own internal Raft-based consensus implementation.
-

Answer Key

This answer key shows the correct option letter after randomization. The complete correct answer text is provided for reference and grading.

Q1. Correct Answer: D

Distributed systems lack shared memory and a global clock, making it impossible to use traditional local locking mechanisms. Furthermore, messages can be lost or delayed, and processes can crash, requiring robust algorithms to ensure safe resource access.

Q2. Correct Answer: B

The centralized approach is simple to implement because a single coordinator grants all locks. However, it suffers from being a single point of failure; if the coordinator crashes, the entire locking system fails.

Q3. Correct Answer: D

In this approach, a token circulates through the processes in a logical ring, and only the holder of the token can access the resource. This ensures fairness and prevents starvation, though the loss of the token is a significant failure state.

Q4. Correct Answer: C

The Ricart-Agrawala algorithm is a fully distributed, timestamp-based voting approach. It requires $2(N-1)$ messages for every request to enter a critical section, where N is the number of processes.

Q5. Correct Answer: A

The Bully algorithm is a classic leader election method based on process identifiers. In this "intuition-only" model, the process with the highest unique ID is elected as the leader.

Q6. Correct Answer: C

Because messages are often in flight, taking snapshots at different wall-clock times can make money or data appear to disappear or double-count. This occurs because the snapshots do not represent a single logical moment across the entire distributed system.

Q7. Correct Answer: C

Markers are sent on all outgoing channels to initiate a snapshot; when a process receives its first marker, it records its local state. It then begins recording messages on other channels until markers have been received on all channels, completing the consistent cut.

Q8. Correct Answer: D

ZooKeeper was originally developed at Yahoo to provide reliable coordination for large-scale distributed systems. It was designed to handle tasks like task assignment and configuration management.

Q9. Correct Answer: C

ZooKeeper utilizes "znodes" (a hierarchical namespace similar to a file system) and "watches" (a mechanism to notify clients of changes). Together, these allow developers to build complex coordination logic.

Q10. Correct Answer: D

Historically, Kafka relied on ZooKeeper for managing its coordination and metadata. In modern implementations, Kafka has largely moved away from ZooKeeper in favor of KRaft (Kafka Raft), its own internal Raft-based consensus implementation.